COMP30027 Assignment 2 Report

Minh Quan Le
Student ID : 1450342

1 Introduction

This experiment is exploring how different machine learning models and image pre-processing techniques can be combined to improve the classification of traffic signs in the German Traffic Sign Recognition Benchmark (GTSRB) dataset. The goal is to see how traditional models like Support Vector Machines (SVM) and Multinomial Logistic Regression compare with more complex models like Convolutional Neural Networks (CNNs), and how different preprocessing methods can impact the results.

1.1 Data Analysis

In the Traffic Sign Recognition Benchmark dataset, there are 5,488 training instances and 2,353 testing instances distributed across 43 distinct labels. As shown in **Appendix 6.1**, the label distribution is notably imbalanced. Speed limit and priority traffic signs make up a substantial portion of the dataset, whereas other signs—such as general caution, slippery road, and end of no passing are significantly underrepresented. This imbalance mirrors real world conditions, where speed limit signs are encountered far more frequently than other types of traffic signs.

2 Simple Classifier

In this section, several feature engineering techniques are introduced alongside a selection of simple learning algorithms, which serve as baseline benchmarks for comparison in the experimental evaluation

2.1 Evaluation

In the subsequent experiment, the performance of these methods is assessed using 5-fold cross-validation to identify potential biases and variations in their results.

2.2 Feature Extraction

In this experiment, the model's classification accuracy is assessed using three distinct feature representations: the original RGB images, their corresponding Histogram of Oriented Gradients (HOG) descriptors, and a composite representation that integrates both. The HOG transformation captures local edge orientations and gradient intensity patterns, which are particularly effective for delineating and distinguishing the structural features characteristic of traffic sign shapes (Xiao et al., 2017). The original images were resized to 32×32 pixels and represented in RGB format, yielding flattened feature vectors of 3072 dimensions. In contrast, the HOG-transformed images were converted into feature vectors of 1024 dimensions. Before training the model, standard pre-processing procedures—including feature normalization—were applied to both representations to ensure consistency and enhance model performance.

2.3 Feature Selection

Given the high dimensionality of the training data, author used two complementary dimensionality reduction methods, principal component analysis (PCA) and mutual information (MI) to improve computational efficiency without sacrificing predictive performance. PCA captures the principal axes of variation in the feature space, while MI selects features that maximize dependency on the target variable. In both cases,author constrained the output to 300 components to balance model complexity and discriminative power.

2.4 Multinomial Logistic Regression

Logistic regression is a classification algorithm that estimates the posterior probability of a class given an input by modeling the probability through regression. Given the presence of 43 target classes, this experiment adopts the multinomial logistic regression approach to handle the multiclass classification task. The number of training iterations is set to 200 to ensure convergence.

Metrics	Original	HOG	Original + HOG
Accuracy	87.02%	75.12%	90.12%
Precision	87.86%	76.37%	90.61%
Recall	87.02%	75.12%	90.12%
F1 Score	87.05%	75.16%	90.09%

Table 1: Performance of Multinomial Logistic Regression using full features.

Metrics	Original	HOG	Original + HOG
Accuracy	86.62%	75.34%	89.50%
Precision	87.37%	76.51%	90.08%
Recall	86.62%	75.34%	89.50%
F1 Score	86.64%	75.34%	89.51%

Table 2: Performance of Multinomial Logistic Regression using PCA extracted features

Metrics	Original	HOG	Original + HOG
Accuracy	91.16%	76.94%	93.58%
Precision	91.73%	77.83%	93.97%
Recall	91.16%	76.94%	93.58%
F1 Score	91.17%	76.92%	93.58%

Table 3: Average performance of Multinomial Logistic Regression using MI extracted features

2.5 Support Vector Machine

Support Vector Machine (SVM) is a non-probabilistic classification algorithm that aims to identify the optimal decision boundary-separating classes within a dataset. It is particularly useful when there is limited prior knowledge about the feature space. In this experiment, the Radial Basis Function (RBF) kernel is employed due to its capability to model complex nonlinear patterns and its robustness against overfitting.

Metrics	Original	HOG	Original + HOG
Accuracy	84.00%	80.22%	87.35%
Precision	85.19%	81.64%	88.41%
Recall	84.00%	80.22%	87.35%
F1 Score	83.91%	80.09%	87.29%

Table 4: Average performance of SVM using full features.

Metrics	Original	HOG	Original + HOG
Accuracy	84.09%	80.22%	87.09%
Precision	85.27%	81.64%	88.16%
Recall	84.09%	80.22%	87.09%
F1 Score	83.99%	80.09%	87.06%

Table 5: Average performance of SVM using PCA extracted features

Metrics	Original	HOG	Original + HOG
Accuracy	93.29%	82.27%	94.44%
Precision	93.68%	83.42%	94.75%
Recall	93.29%	82.27%	94.44%
F1 Score	93.24%	82.25%	94.43%

Table 6: Average performance of SVM using MI extracted features

2.6 Overall Analysis

The results indicate that both Support Vector Machine (SVM) and Multinomial Logistic Regression classifiers exhibit enhanced performance when trained on feature vectors incorporating both RGB and Histogram of Oriented Gradients (HOG) descriptors. This outcome is expected, as the fusion of color and structural information facilitates the learning of more comprehensive and discriminative feature representations. Regarding dimensionality reduction techniques, Mutual Information (MI) consistently outperforms Principal Component Analysis (PCA), suggesting a stronger statistical dependency between the selected features and the target labels in the case of MI. This advantage is well-founded, as MI is specifically designed to capture features that are highly informative with respect to the output variable—a crucial factor in traffic sign recognition, where distinct sign categories are often defined by salient visual patterns. This interpretation is further substantiated by the test accuracy results obtained on the Kaggle platform, as presented in the following table.

Method	Accuracy
SVM - MI - (Original + HOG)	95.09%
LOG - MI - (Original + HOG)	93.69%

Table 7: Accuracy of highest performing method on Kaggle

Given the improved performance demonstrated by the Support Vector Machine (SVM) among the classifiers evaluated, subsequent analyses focus on the SVM model applied to images preprocessed using Histogram of Oriented Gradients (HOG) and RGB feature transformations, followed by feature selection through Mutual Information. This approach achieved the highest classification accuracy within the scope of this study, as evidenced by k-fold cross-validation results. The confusion matrix presented in Appendix 6.3 indicates that the model effectively captures the shape-based characteristics of the majority of traffic sign classes. However, a slight

decrease in performance is observed for numerically labeled signs, such as speed limits, as well as for more complex categories including general caution (label 18) and wild animal crossing (label 31). The observed misclassification patterns suggest that the model predominantly relies on shape features and is less capable of recognizing more complex traffic signs, implying that the current feature engineering strategy may lack sufficient representational power to capture the subtle patterns of the images.

Figure 1: Speed Limit Sign misclassified

Figure 2: Complex Traffic Sign misclassified

3 Convolutional Neuron Network

In this section, the author trains convolutional neural networks (CNNs), which are known for their efficiency in image classification tasks. CNNs require fewer learnable parameters compared to traditional multilayer perceptron (MLP) networks, resulting in lower memory usage and an improvement in computational efficiency while being able to maintain performance.

3.1 Image Preprocessing

Prior to the training process, all images were converted to RGB channels and resized to a resolution of 32×32 pixels. In addition, training images were normalized to the range [0, 1] to help the model learn faster and perform better.

3.2 Model Architecture

3.2.1 AlexNet 1

The first convolutional neural network (CNN) architecture was inspired by AlexNet (Krizhevsky et al., 2012), which is known for its effectiveness in the processing of color images. This model comprises six learnable layers: four convolutional layers that extract local features from the input images, such as edges and corners, followed by two fully connected layers that learn high-level representations from the extracted features, observe **Appendix 6.7**. The Rectified Linear Unit (ReLU) activation function was employed throughout the network due to its capacity to accelerate convergence during training by mitigating vanishing gradient issues. Furthermore, a Dropout layer with a dropout rate of 0.5 was incorporated to reduce the risk of overfitting by randomly deactivating neurons during training (detail in **Appendix 6.4**).

3.2.2 AlexNet 2

The second architecture is similar to the first one, except that in this version, the author introduces batch normalization layers into the proposed network to address brightness variations in the input images and to improve training speed (Fang et al., 2022). Batch normalization layers learn the mean and variance of the dataset, standardizing the input distributions for each layer. By normalizing the mean and variance of the feature maps, batch normalization reduces variations in image brightness, thereby enhancing the network's ability to detect low-brightness instances and improving overall performance (Fang et al., 2022) (detail in **Appendix 6.4**).

3.3 Data Augmentation

Convolutional Neural Networks (CNNs) are characterized by a large number of learnable parameters, which makes them particularly susceptible to overfitting, especially in scenarios where the available training data is limited. To mitigate this risk and enhance the generalization capability of the model, various data augmentation techniques were implemented to artificially increase the variability of the training dataset (Krizhevsky et al., 2012). The augmentation strategies employed include minor image rotations—applied cautiously to avoid semantic alterations of the image content—as well as translation (shifting) and shearing transformations. Examples of the augmented data can be

found in the **Appendix 6.2**.

3.4 Training and Result

To evaluate the model's performance, the author employed 5-fold cross-validation and used accuracy, precision, and recall as the primary evaluation metrics. The number of training epochs was fixed at 60. Weighted averaging strategy was applied to compute overall precision, recall and f1-score. Model training was conducted using a fixed learning rate of 0.001 with the Adaptive Moment Estimation (ADAM) optimizer. This configuration was chosen to reduce training oscillations and facilitate faster convergence toward an optimal solution.

Metrics	Original data	Augmented data
Accuracy	98.14%	99.36%
Precision	98.26%	99.40%
Recall	98.14%	99.36%
F1 Score	98.13%	99.36%

Table 8: Average performance of AlexNet1.

Metrics	Original data	Augmented data
Accuracy	98.72%	99.23%
Precision	98.84%	99.27%
Recall	98.72%	99.23%
F1 Score	98.72%	99.22%

Table 9: Average performance of AlexNet2.

Model	Accuracy
AlexNet1	99.69%
AlexNet2	99.39%

Table 10: Kaggle test accuracy for each model with data augmentation.

3.5 Analysis

The experimental results underscore the efficacy of convolutional neural networks (CNNs) in addressing image classification tasks. Both CNN architectures exhibited robust performance when evaluated through cross-validation and on the Kaggle test dataset.

Notably, in the absence of data augmentation, both models demonstrated substantial overfitting to the training data, indicating limited generalization capability. The incorporation of data augmentation significantly enhanced model performance, underscoring its critical role in promoting generalization. Consequently, the subsequent analysis assumes data augmentation as a foundational component of the training pipeline.

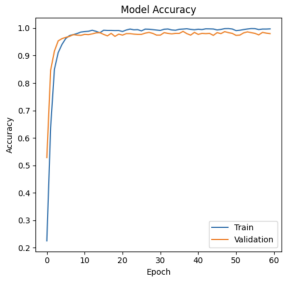

Figure 3: Overfitting: AlexNet1 with original data

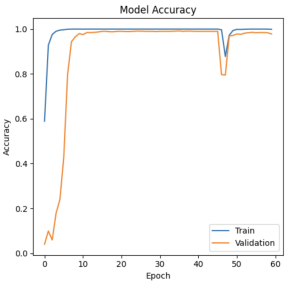

Figure 4: Overfitting: AlexNet2 with original data

A comparative evaluation of the two architectures reveals that AlexNet1 exhibits better generalization relative to AlexNet2 (**Figure 5 & Figure 6**). The more significant gaps between training and validation accuracies observed in AlexNet2 suggests a greater degree of overfitting in the early stages of training. This instability may be attributed to the influence of batch normalization layers, which can accelerate learning in and cause the model to initially overfit on low-level features such as brightness. Additionally, the validation set may contain images with brightness distributions that differ from those in the training set. As training progresses, AlexNet2 gradually learns more discriminative features, resulting in improved validation performance and enhanced generalization.

For the purpose of critical analysis, author selects the performance of AlexNet1 with data augmentation given the generalization capability. Overall, the model achieves accuracies of 99.36% and 99.69% on the validation and Kaggle test sets, respectively. Upon examining instances where the model produced incorrect classifications, it is observed that misclassifications predominantly occur with images that are noisy, excessively blurred (**Figure 8**), or overly dark conditions (**Figure 7**).

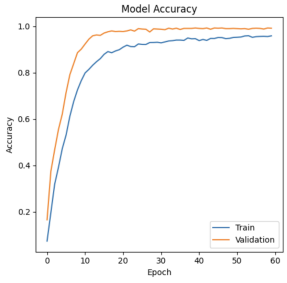

Figure 5: Accuracy over epochs for AlexNet1 with data augmentation

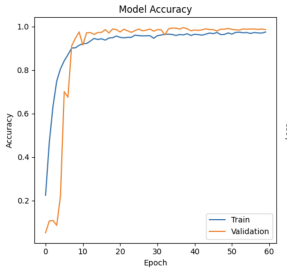

Figure 6: Accuracy over epochs for AlexNet2 with data augmentation

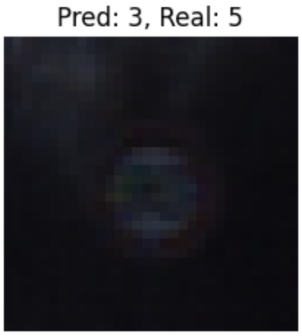

Figure 7: Overly dark image misclassified by AlexNet1

Figure 8: Blurry image misclassified by AlexNet1

4 Ensemble Method

By considering the limitations of Convolutional Neural Networks (CNNs) discussed in Section 3. In the subsequent section, a range of image pre-processing techniques are employed, with each processed image subset allocated to a distinct Convolutional Neural Network (CNN) functioning as an expert. The individual predictions generated by these expert models are then aggregated and utilized to train a meta-classifier, thereby aiming to improve overall classification performance through ensemble learning.

4.1 Image Pre-processing

4.1.1 Bilateral Filters

Due to the less effective performance of the CNN model when applied to noisy image data, the author incorporates bilateral filtering as a pre-processing step. Bilateral filters are effective in denoising images by smoothing homogeneous regions while preserving edge information (Paris et al., 2007). This facilitates the extraction of salient features, such as edges, enabling the model to focus on meaningful patterns without being adversely affected by noise.

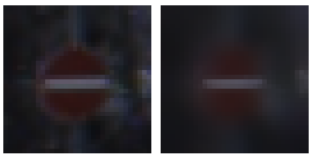

Figure 9: Image before and after applied to Bilateral Filters

4.1.2 Gamma Correction

To address the challenge posed by overly dark images during classification, the author applied Gamma Correction—a nonlinear intensity transformation technique that adjusts image brightness and contrast (Rosebrock, 2015). This method modifies pixel intensities according to a power-law expression, thereby enhancing visibility in underexposed regions without significantly distorting the overall image content. The application of gamma correction offers several advantages for Convolutional Neural Networks (CNNs). It helps normalize lighting conditions across the dataset, introduces greater variability in brightness and contrast, and makes features in shadowed areas more detectable.

4.2 Architecture

Following the pre-processing stage, the images are categorized into three distinct sets: original images, bilateral filtered images, and gamma-corrected images. Each set is subsequently

Figure 10: Image before and after applied to Gamma Correction

used to train an individual Convolutional Neural Network (CNN), enabling each model to specialize in learning discriminative features specific to its corresponding image distribution. This ensemble approach treats each CNN as an expert model, trained to capture variations introduced by different pre-processing techniques. The outputs of these expert models are then integrated using a meta-classifier; in this study, Support Vector Machine (SVM) and Multinomial Logistic Regression are employed for this purpose. The rationale behind this methodology is to enhance the overall generalization capability of the system, particularly in scenarios involving image degradation due to blurring or variations in brightness. A detailed illustration of the proposed model architecture is provided in **Appendix 6.6**.

4.3 Training and Result

To conduct the experiments, the author employs AlexNet1, selected for its generalization capabilities within the constraints of the available data. The training configuration for AlexNet1 is maintained consistently with the settings described in Section 3. As previously noted, Logistic Regression and Support Vector Machine (SVM) are utilized as meta-classifiers to generate the final predictions by aggregating the outputs from the expert CNNs. 5-fold cross-validation strategy was applied for performance evaluation.

Metrics	SVM	Multi Logistic Regression
Accuracy	99.63%	99.61%
Precision	99.64%	99.63%
Recall	99.63%	99.61%
F1 Score	99.63%	99.61%

Table 11: Average performance of Ensemble Method with different meta learners

4.4 Analysis

The ensemble approach showed modest improvements across all evaluation metrics compared to base CNN models in section 3, achieving an accuracy of **99.79%** on Kaggle when using SVM and logistic regression as meta-classifiers. This outcome aligns with expectations, as ensemble methods can capture a wider range of data patterns, leading to enhanced generalization. However by judging on the misclassified cases, the method still struggles to accurately classify images that are heavily disrupted.

Figure 11: Overly dark image correctly classified by the method

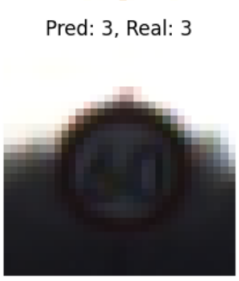

Figure 12: Blurry image correctly classified by the method

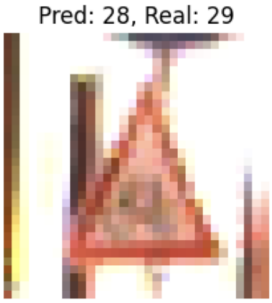

Figure 13: Disrupted image misclassified by the method

5 Conclusions

In the experiment, the authors explored various machine learning algorithms and techniques to enhance model performance. While simpler methods were effective in classifying ba-

sic traffic signs, they struggled with more complex signs—an area where convolutional neural networks (CNNs) performed significantly better. To address the challenge of limited training data, the authors employed several data augmentation techniques. As a result, the CNN models achieved excellent performance, with all evaluation metrics reaching around 99%. In conclusion, for further improvement, it is important to collect more data and ensure that the images used are not disrupted or degraded.

References

Hai-Feng Fang, Jin Cao, and Zhi-Yuan Li. 2022. A small network micronnet-bf of traffic sign classification. *Computational Intelligence and Neuroscience*, 2022:e3995209, 03.

Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. Imagenet classification with deep convolutional neural networks. *Communications of the ACM*, 60:84–90, 05.

Sylvain Paris, Pierre Kornprobst, Jack Tumblin, and Fr edo Durand. 2007. A gentle introduction to bilateral filtering and its applications.

Adrian Rosebrock. 2015. Opencv gamma correction, 10.

Zhitao Xiao, Zhenjie Yang, Lei Geng, and Fang Zhang. 2017. Traffic sign detection based on histograms of oriented gradients and boolean convolutional neural networks. pages 111–115, 02.

6 Appendix

6.1 Class Distribution

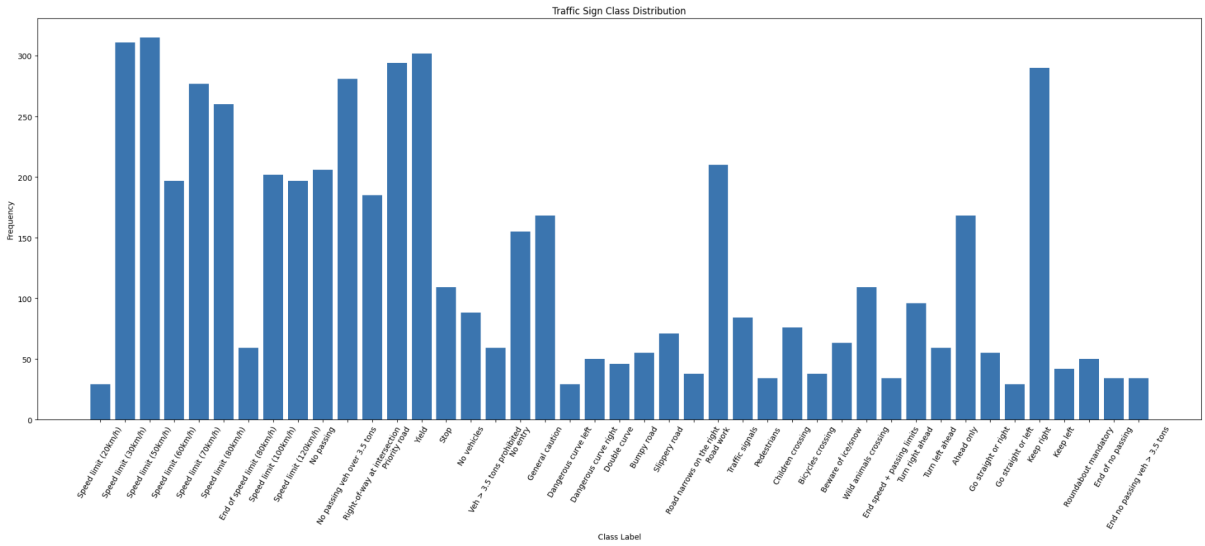

Figure 14: Label distribution of the training set

6.2 Sample Data Augmentation

Figure 15: Sample Data Augmentation

6.3 Confusion matrix for SVM - MI - (Original + HOG)

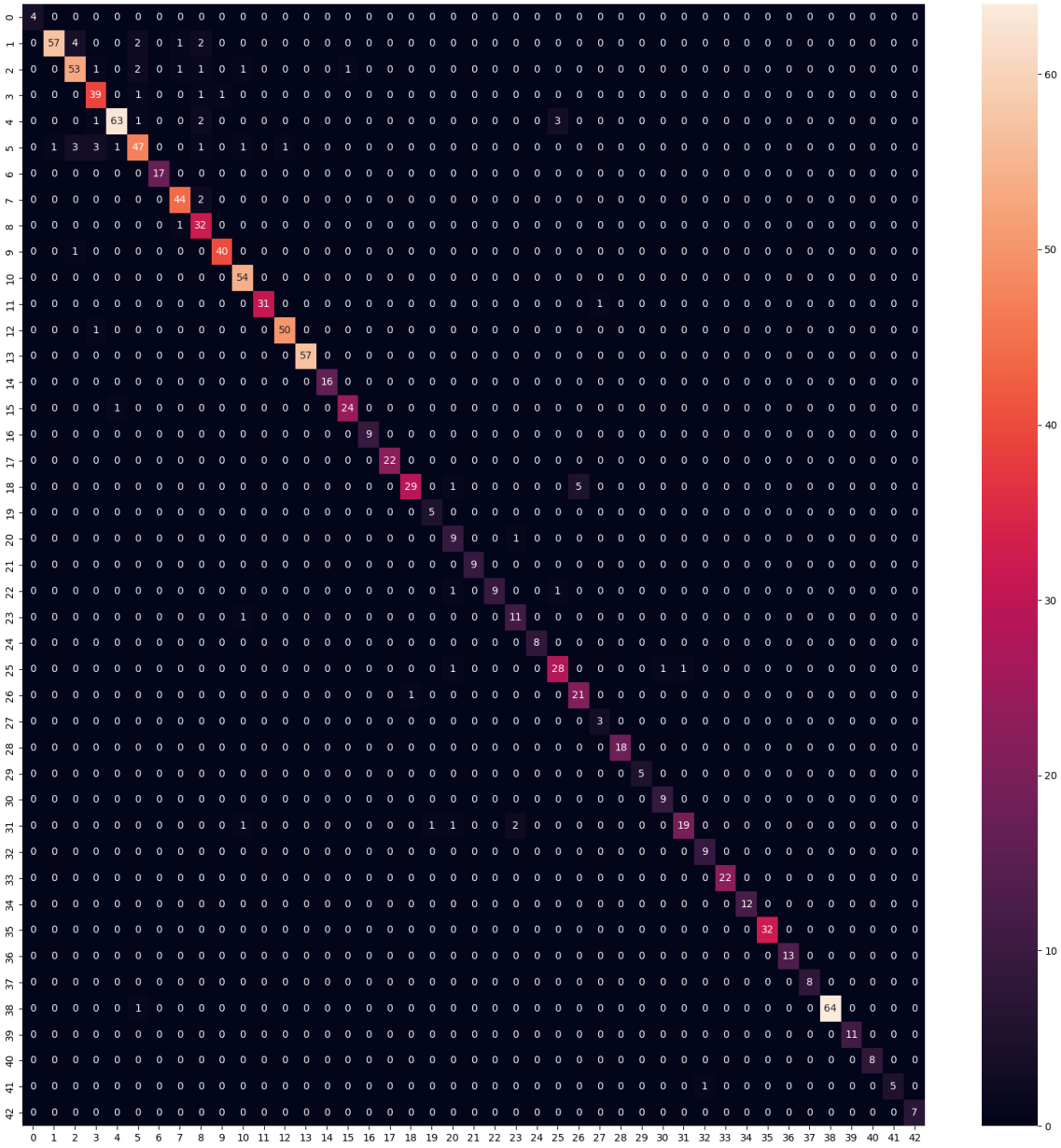

Figure 16: SVM - MI - (Original + HOG) confusion matrix

6.4 CNN Architecture Diagram

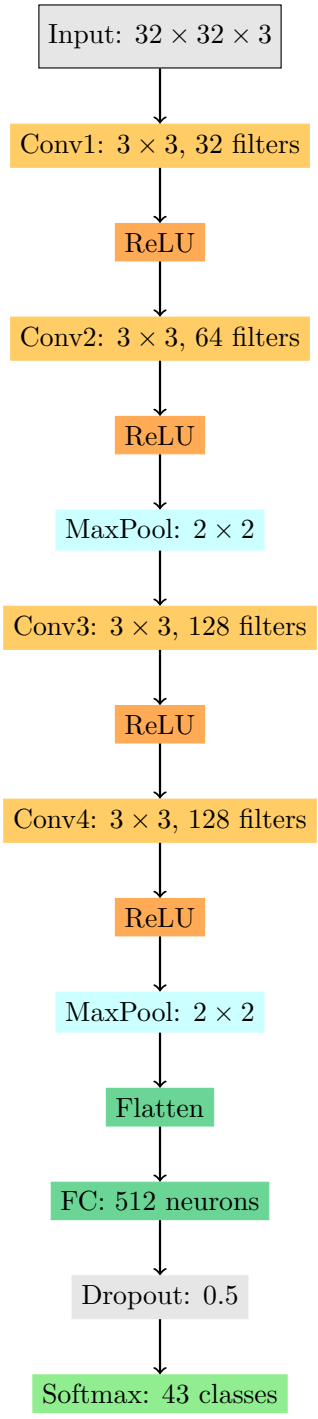

Figure 17: The architecture of AlexNet1

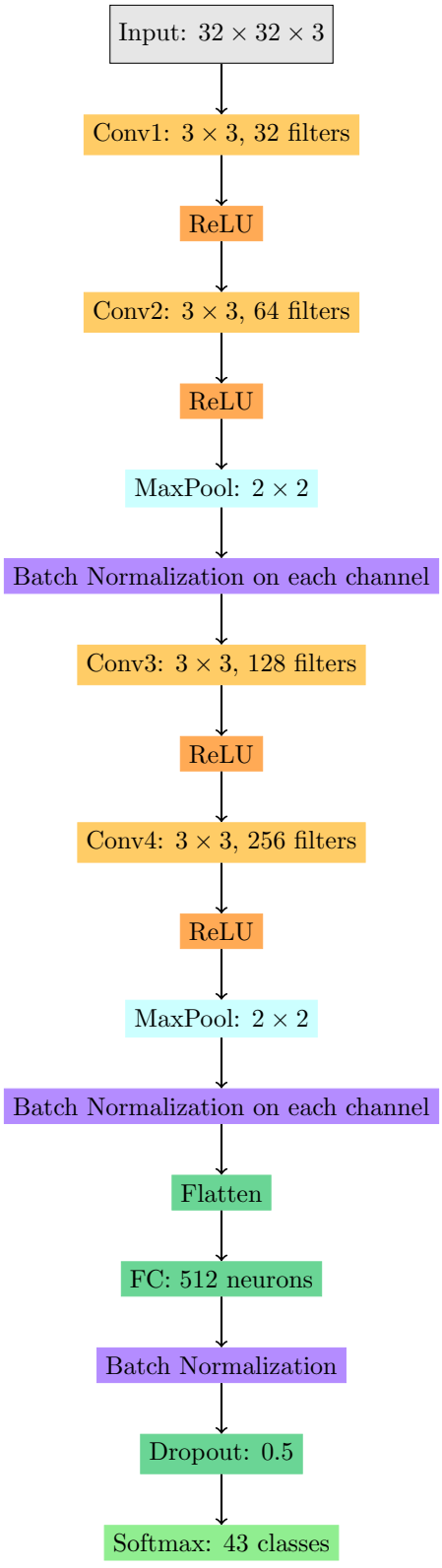

Figure 18: The architecture of AlexNet2

6.5 Confusion matrix for Section 3 Models

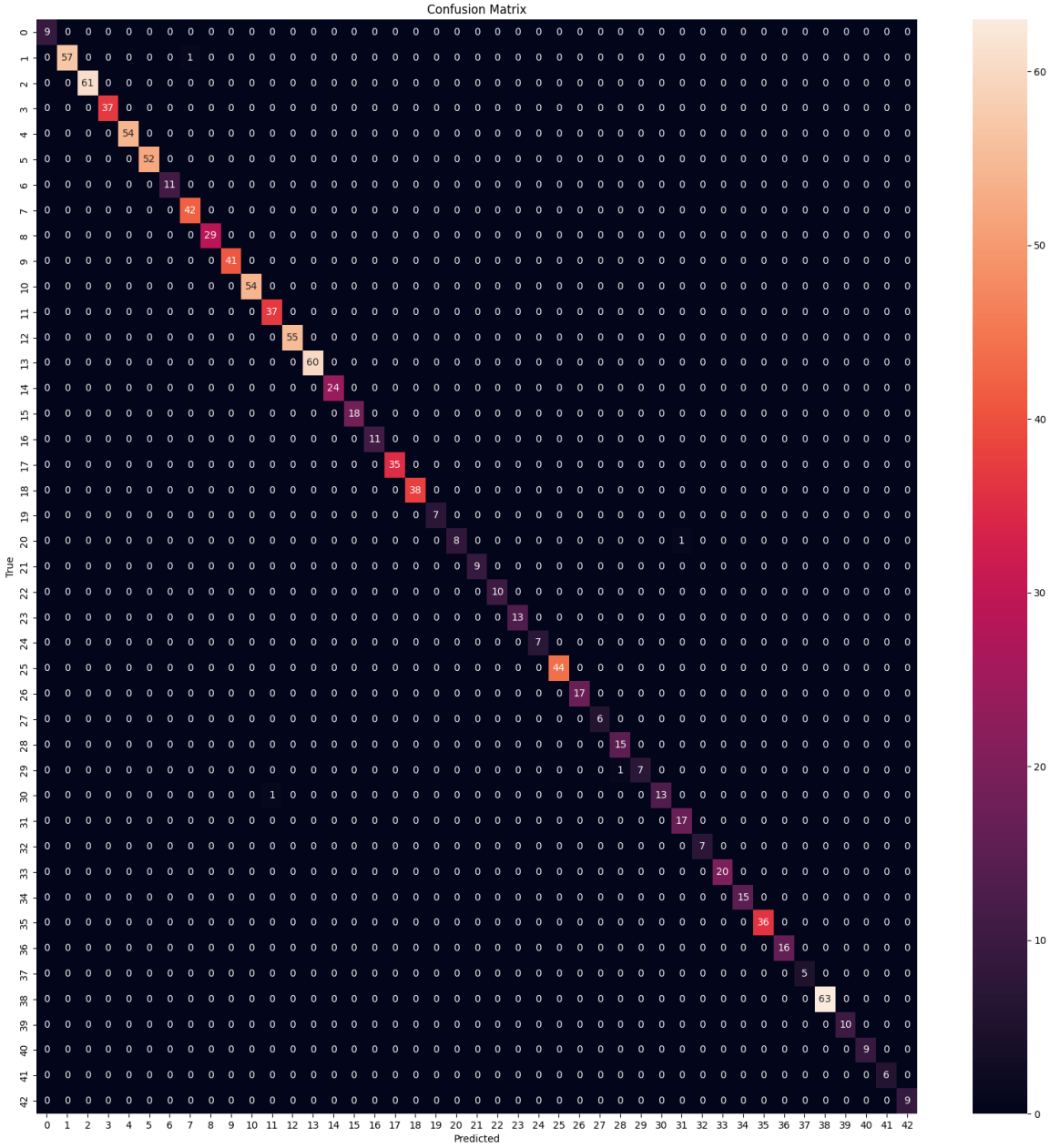

Figure 19: Confusion Matrix for AlexNet1

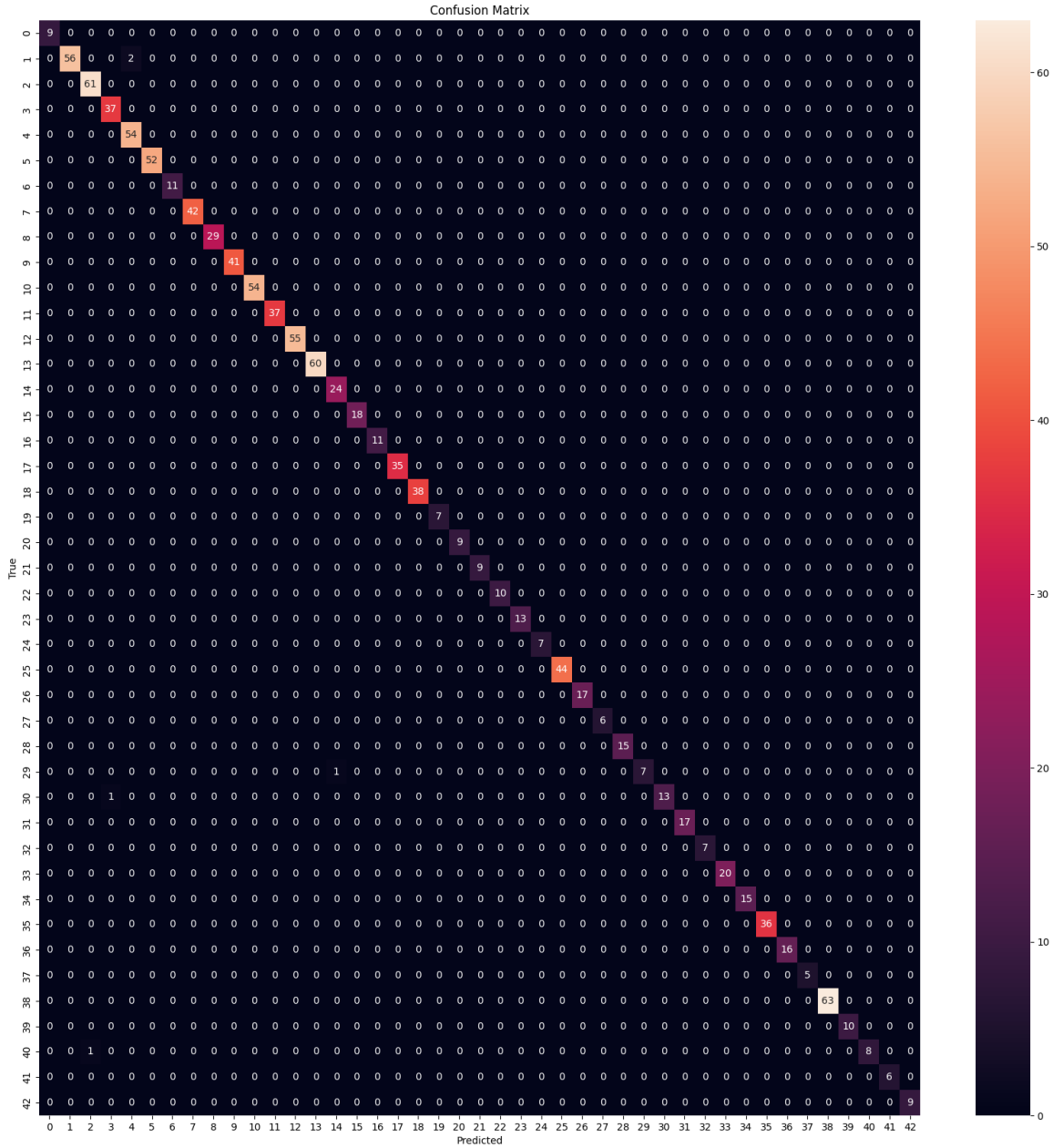

Figure 20: Confusion Matrix for AlexNet2

6.6 Ensemble method Architecture

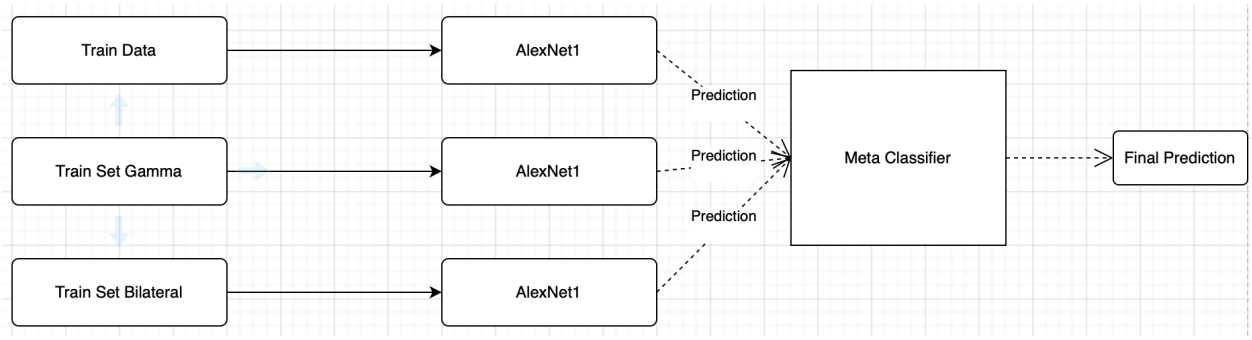

Figure 21: The architecture of Ensemble Method

6.7 CNN Visualization

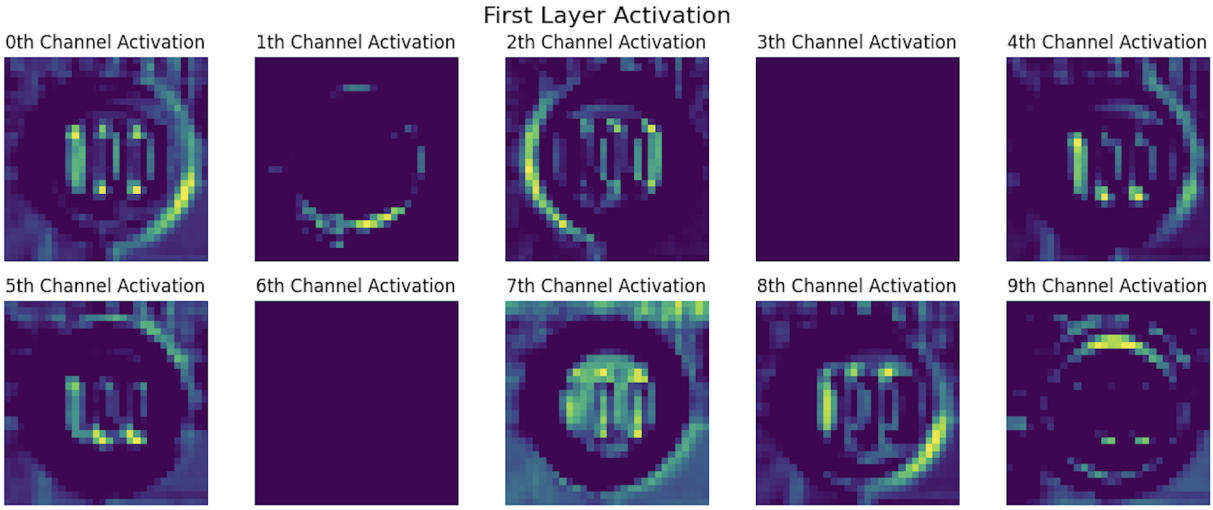

Figure 22: Visualization of the first 10 channels of the first layer of AlexNet1

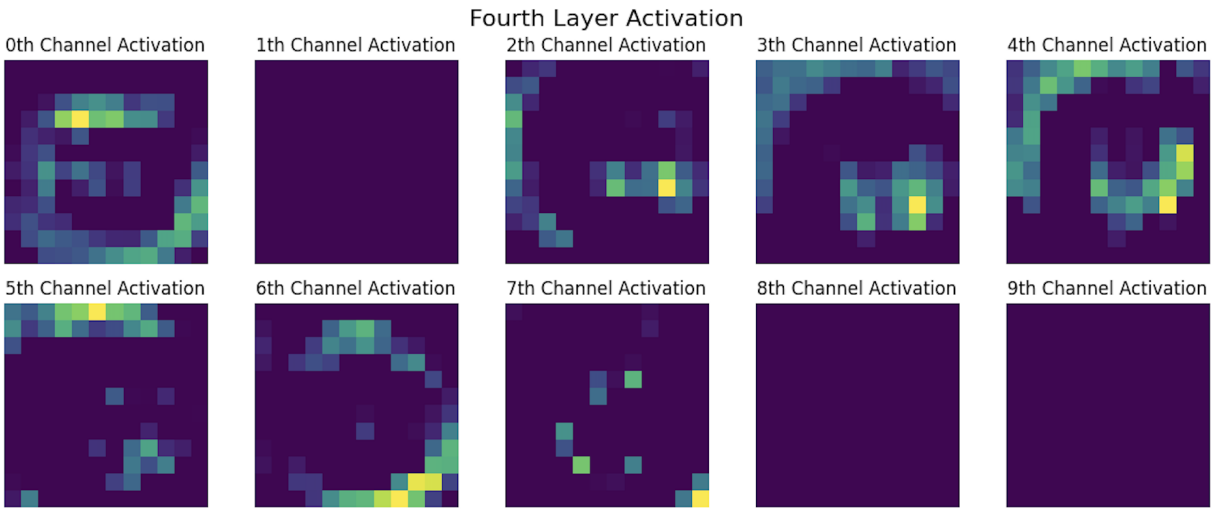

Figure 23: Visualization of the first 10 channels
of the fourth layer of AlexNet1

6.8 Confusion Matrix for Ensemble Method

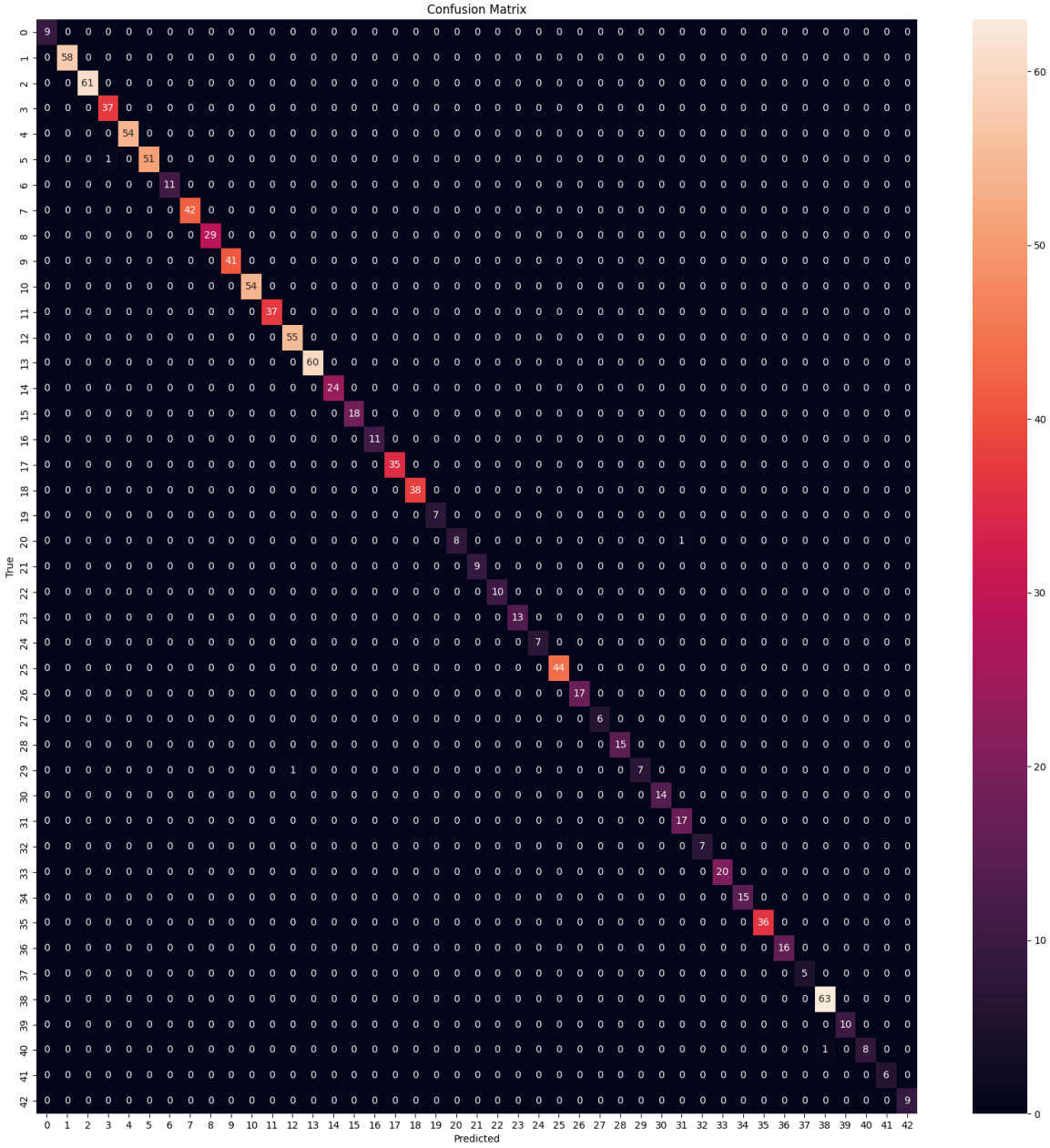

Figure 24: Confusion Matrix for Ensemble Method